

Quantum Swarm

An Autonomous Multi-Agent System That
Builds Tested Codebases End-to-End

Team Members

- **Luka Mikautadze**
- **Rezi Darsavelidze**
- **Giorgi Siradze**
- **Davit Mzhavanadze**



Problem and User

Target User

Software developers, technical leads, and founders needing rapid, reliable MVPs.

Pain 1: Setup Friction

Days wasted on database schema boilerplate, basic testing configurations, and API structures before writing core logic.

Pain 2: Fragmented Outputs

Existing LLM code-generation helpers output isolated, buggy files that lack systemic coordination and fail compilation.

Why It Matters Now

Market speed demands instant iterations, but manual coordination and standard code-completion tools do not produce complete, integrated architectures.

What It Does

Core Capacity

Single-prompt autonomous codebase generation.

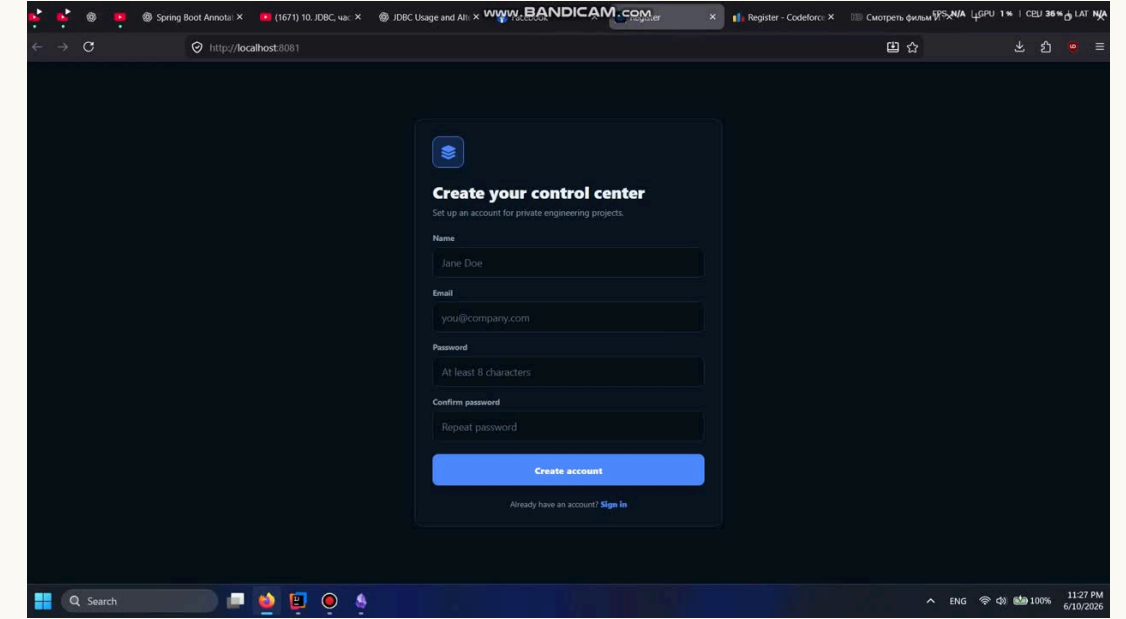


The Sequential Sprint Process

- **CEO Agent:** Plans the sprint goals based on the user's brief.
- **Architecture Team:** Defines system structures, APIs, and database schemas.
- **Design Team:** Configures UI flows, component specifications, and style.
- **Engineering Team:** Writes actual, runnable code using 8 parallel specialist agents.
- **QA Team:** Runs compilers, linters, and verification test suites.

Live Demo

Scripted Happy Path Flow



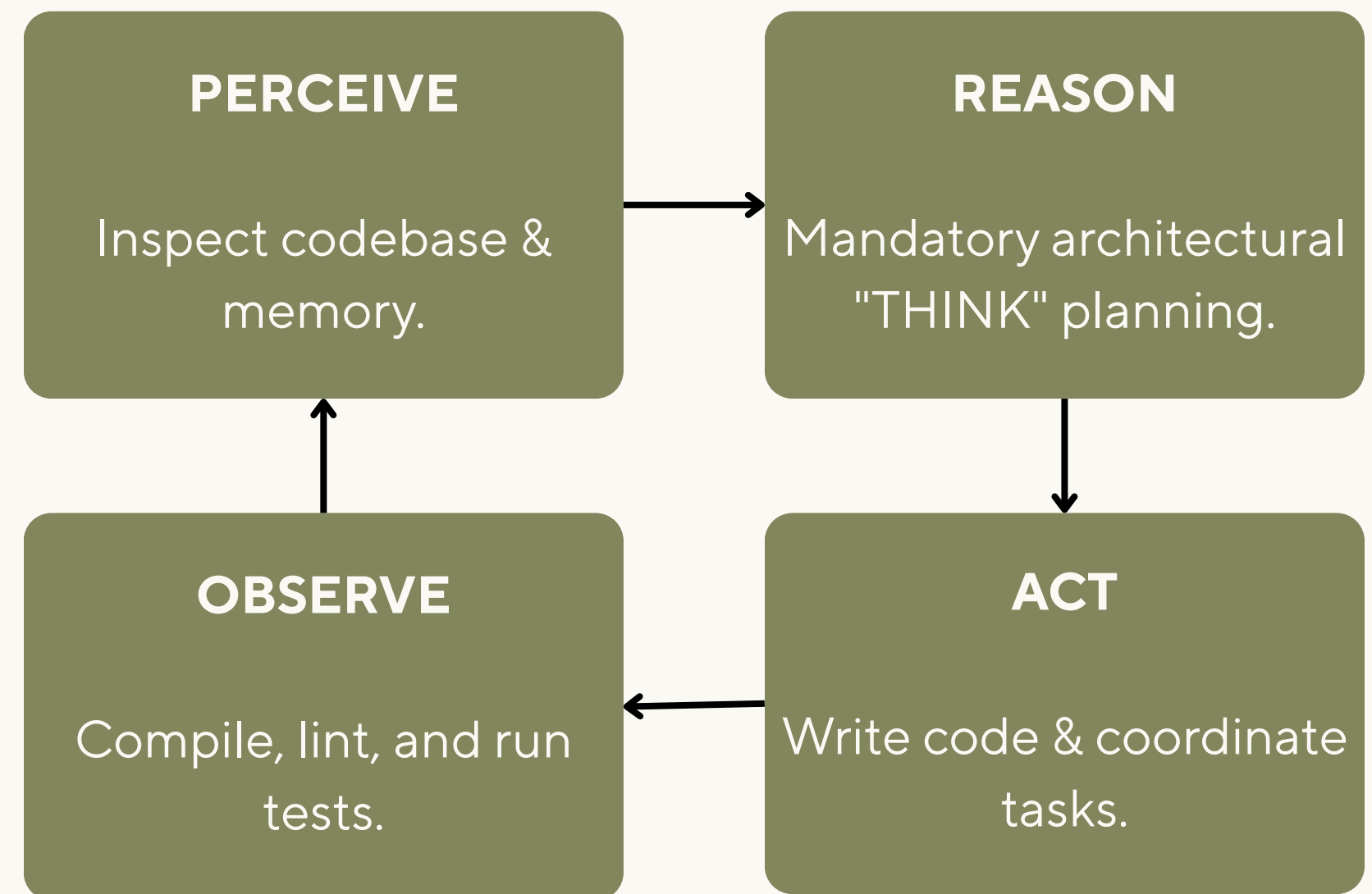
- **CEO Coordination:** Watch the supervisor agent partition tasks to the sub-teams.
- **Parallel Engineering:** See 8 developer agents build code inside isolated git worktrees to prevent merge conflicts.
- **Automated Testing:** Watch the QA agent compile the code and execute unit tests.
- **Clean Delivery:** Open `/company_output/code/` to show the fully populated, runnable codebase.

Architecture

Architecture & Stack

- **Core Stack:** Python 3.11, SQLite, and NetworkX (Graph RAG).
- **Worktree Control:** Git isolation for 8 parallel developer agents.
- **Models:** Gemini 3.1 Flash (Main logic) + Vision helpers.
- **System Fallback:** Circuit breaker halts execution after 3 consecutive failures.

The Loop Flow



Measurements

REAL EVALUATION NUMBERS

- **Golden-Set Score:** 10 out of 10 passing scores on canonical evaluation tests.
- **P95 Latency:** 7.94 seconds (calculated from the sorted latency distribution of the successful calls, excluding the rare 60-second timeouts that triggered resilient retries).
- **Average Cost per LLM Call:** \$0.0028 (approximately 0.28 cents per request, highlighting the high efficiency of the Gemini 3.1 Flash model setup).
- **Load-Test Result:** Successfully managed 8 parallel developer agents writing and merging code simultaneously without Git conflicts.

Safety and reliability

Threat Model & Mitigations

- **Attacks Run:** Executed automated scripts testing cross-user isolation boundaries and unauthorized workspace directory manipulation.
- **Guardrails That Held:** Bearer-token authentication, Pydantic schemas for parameter verification, and mandatory THINK steps checked before irreversible tools (`write_code_file`, `run_shell`) run.
- **Failure Path:** Application sanitizes and filters all internal error traces, showing clean, safe messages to the API layer.

Lessons and Next

Key Retrospective Lesson

KEY RETROSPECTIVE LESSON	FUTURE ROADMAP
Vector RAG Limitation: Pure vector similarity matches keywords but misses system-wide architecture logic. Implementing Graph RAG with spreading activation was necessary to connect past engineering design lessons (such as identifying database connection leaks).	Next Implementation: Build multi-repository synchronization support and native Model Context Protocol (MCP) servers for external tool integration.